

编程任务：基于 Vibe Coding 的 CIFAR-10 图像分类训练系统

一、任务背景

本任务面向电子信息、图像处理、深度学习方向的一年级研究生，要求使用 vibe coding 的方式完成一个完整的图像分类训练项目。

所谓 vibe coding，是指你不直接手写代码，而是通过自然语言向 AI 描述需求、反馈错误、提出修改要求，并通过运行、测试、检查结果来推动项目逐步完成。

本任务要求你在 CIFAR-10 数据集上训练 ResNet-18 图像分类模型，并完成完整的训练、验证、测试、日志记录和代码仓库管理流程。

二、任务目标

你需要通过自然语言驱动 AI 生成一个可运行的 Python 深度学习项目，实现以下目标：

1. 自动下载或读取 CIFAR-10 数据集；
2. 构建 ResNet-18 图像分类模型；
3. 实现完整训练流程；
4. 实现验证流程；
5. 实现测试流程；
6. 使用 W&B 或 TensorBoard 记录训练曲线；
7. 保存模型权重；
8. 输出测试集分类准确率；
9. 生成实验结果报告；
10. 将完整代码推送到 Git 仓库。

三、任务限制

1. 不允许手写核心代码。
2. 可以阅读 AI 生成的代码。
3. 可以运行代码、查看报错、定位问题。
4. 修改代码应优先通过自然语言要求 AI 完成。
5. 允许通过自然语言要求 AI 解释代码、重构代码、修复 bug。
6. 项目应优先支持 CPU 运行；如果有 GPU，可以自动使用 GPU。
7. 不要求追求最高准确率，重点是完整流程、工程规范和实验记录。

四、基础功能要求

4.1 数据集处理

项目需要支持 CIFAR-10 数据集的加载和预处理。

要求包括：

1. 自动下载 CIFAR-10；
2. 划分训练集、验证集和测试集；
3. 对训练集进行必要的数据增强；
4. 对验证集和测试集进行标准化处理；
5. 支持设置 batch size；
6. 支持设置 num_workers；
7. 能够在 CPU 环境下运行。

建议的数据增强包括：

1. 随机裁剪；
2. 随机水平翻转；
3. 图像归一化。

4.2 模型构建

项目需要构建 ResNet-18 模型。

要求包括：

1. 使用 PyTorch 实现；
2. 支持使用 torchvision 中的 ResNet-18；
3. 修改最后一层分类头，使其适配 CIFAR-10 的 10 个类别；
4. 支持从头训练；
5. 可选支持 ImageNet 预训练权重。

4.3 训练流程

项目需要实现完整训练流程。

要求包括：

1. 支持设置训练轮数；
2. 支持设置学习率；
3. 支持设置优化器；
4. 支持设置随机种子；
5. 每个 epoch 输出训练损失和训练准确率；
6. 每个 epoch 后进行验证；
7. 保存验证集表现最好的模型；
8. 支持断点保存；
9. 训练完成后保存最终模型。

4.4 验证流程

项目需要实现验证集评估。

要求包括：

1. 每个 epoch 后在验证集上评估；

2. 输出验证损失；
3. 输出验证准确率；
4. 根据验证准确率保存最佳模型；
5. 能够发现过拟合趋势。

4.5 测试流程

项目需要实现独立测试流程。

要求包括：

1. 加载最佳模型；
2. 在测试集上评估；
3. 输出测试准确率；
4. 输出每一类的分类准确率；
5. 输出混淆矩阵；
6. 保存测试结果文件。

4.6 日志记录

项目需要至少支持以下一种实验记录方式：

1. TensorBoard；
2. W&B。

日志中至少记录：

1. train loss；
2. train accuracy；
3. validation loss；
4. validation accuracy；
5. learning rate；
6. test accuracy；
7. 训练参数配置。

如果使用 W&B，应避免把 API Key 写入代码文件。

如果使用 TensorBoard，应提供启动 TensorBoard 的命令说明。

4.7 Git 仓库管理

项目需要使用 Git 管理代码。

要求包括：

1. 创建 Git 仓库；
2. 至少保留 5 次有意义的 commit；
3. commit 信息应能反映开发过程；
4. 不应提交大型模型文件、缓存文件和数据集文件；
5. 提供 `.gitignore` 文件；
6. 最终将代码推送到远程 Git 仓库。

五、建议项目结构

最终项目建议采用如下结构：

```
cifar10-resnet18-vibecoding/
├── README.md
├── requirements.txt
├── .gitignore
├── configs/
│   └── default.yaml
├── data/
│   └── README.md
├── checkpoints/
│   └── README.md
├── logs/
│   └── README.md
├── outputs/
│   ├── test_metrics.json
│   ├── confusion_matrix.png
│   └── report.md
├── src/
│   ├── __init__.py
│   ├── dataset.py
│   ├── model.py
│   ├── train.py
│   ├── evaluate.py
│   ├── test.py
│   ├── utils.py
│   └── logger.py
├── scripts/
│   ├── train.sh
│   ├── test.sh
│   └── run_tensorboard.sh
└── docs/
    └── vibe_coding_process.md
```

可以根据实际情况调整结构，但必须保证项目清晰、可运行、可复现。

六、命令行运行要求

项目应至少支持以下命令：

```
python src/train.py --config configs/default.yaml
```

```
python src/test.py --config configs/default.yaml --checkpoint checkpoints/best_model.pth
```

如果使用 TensorBoard，应支持：

```
tensorboard --logdir logs/
```

如果使用 W&B，应在 README 中说明如何登录和查看实验结果。

七、配置文件要求

项目应使用配置文件管理主要参数。

配置文件至少包含：

```
seed: 42
dataset:
  name: CIFAR10
  data_dir: ./data
  batch_size: 128
  num_workers: 2

model:
  name: resnet18
  num_classes: 10
  pretrained: false

training:
  epochs: 10
  learning_rate: 0.001
  optimizer: adam
  weight_decay: 0.0005

logging:
  backend: tensorboard
  log_dir: ./logs

checkpoint:
  save_dir: ./checkpoints
  save_best: true
```

八、实验结果要求

最终结果至少应包括：

1. 训练集 loss 曲线；
2. 验证集 loss 曲线；
3. 训练集 accuracy 曲线；
4. 验证集 accuracy 曲线；
5. 测试集整体准确率；
6. 每一类测试准确率；
7. 混淆矩阵；
8. 简短实验分析。

实验分析至少回答：

1. 模型是否正常收敛？
2. 是否出现明显过拟合？
3. 哪些类别容易被混淆？
4. 如果继续改进，可以从哪些方面入手？

九、Vibe Coding 过程记录要求

你需要在 `docs/vibe_coding_process.md` 中记录完整开发过程。

至少包括：

1. 初始需求提示词；
2. AI 第一次生成的项目结构；
3. 第一次运行遇到的问题；
4. 报错信息；
5. 你如何向 AI 描述错误；
6. AI 如何修改；
7. 修改后是否解决；
8. 后续新增功能的提示词；
9. 最终验收过程；
10. 你对 vibe coding 的总结。

记录格式建议如下：

Vibe Coding 过程记录

第 1 轮：初始需求

我的提示词：

```
```text
```

请帮我生成一个 PyTorch 项目，在 CIFAR-10 上训练 ResNet-18.....

AI 输出摘要：

- 生成了项目结构；
- 生成了训练脚本；
- 生成了 README。

运行结果：

这里粘贴运行命令和结果

遇到的问题：

这里粘贴报错

我的反馈提示词：

运行时报错如下，请帮我分析原因并修改代码……

修改结果：

说明是否解决

## ## 十、建议开发流程

建议按照以下步骤完成任务：

### ### 第一步：生成项目骨架

让 AI 生成完整项目结构、README、requirements.txt、训练脚本和测试脚本。

### ### 第二步：跑通最小训练流程

先设置较小 epoch，例如 1 或 2，确认代码能完整跑通。

### ### 第三步：加入日志记录

让 AI 添加 TensorBoard 或 W&B 记录训练曲线。

### ### 第四步：加入模型保存和测试流程

让 AI 实现 best checkpoint 保存、测试集评估和混淆矩阵输出。

### ### 第五步：完善配置文件

让 AI 将 batch size、learning rate、epoch、日志方式、保存路径等参数放入配置文件。

### ### 第六步：整理 README 和实验报告

让 AI 根据最终代码生成使用说明和实验结果报告模板。

### ### 第七步：整理 Git 提交记录

按功能阶段提交代码，例如：

```
```text
init project structure
add cifar10 dataloader
add resnet18 training pipeline
add tensorboard logging
```

```
add test evaluation and confusion matrix
update readme and experiment report
```

十一、常见问题处理要求

项目应尽量处理以下问题：

1. 没有 GPU 时自动使用 CPU；
2. CUDA 不可用时不报错退出；
3. 数据集下载失败时给出提示；
4. batch size 太大导致内存不足时可以调整；
5. W&B 未登录时给出处理说明；
6. checkpoints、logs、outputs 文件夹不存在时自动创建；
7. 测试脚本找不到模型文件时给出清晰报错；
8. Windows 和 Linux 路径兼容。

十二、提交内容

最终提交以下内容：

1. Git 仓库链接；
2. 完整项目代码；
3. README.md；
4. requirements.txt；
5. 配置文件；
6. 训练日志截图或链接；
7. 测试结果；
8. 混淆矩阵；
9. 实验报告；
10. vibe coding 过程记录。

十三、评分标准

总分 100 分。

模块	分值	评价内容
需求描述与过程记录	20	是否清楚记录 vibe coding 的需求、反馈、调试和迭代过程
项目可运行性	20	是否能完成训练、验证、测试全流程
深度学习流程完整性	20	是否包含数据加载、模型构建、训练、验证、测试、保存模型
日志与可视化	15	是否使用 W&B 或 TensorBoard 记录训练曲线
工程规范	15	是否有清晰项目结构、README、requirements、配置文件、Git 管理
实验分析	10	是否分析训练结果、测试结果、类别混淆和改进方向

十四、加分项

完成以下内容可以酌情加分：

1. 支持命令行参数覆盖配置文件；
2. 支持学习率调度器；
3. 支持早停策略；
4. 支持自动生成 Markdown 实验报告；
5. 支持混淆矩阵可视化；
6. 支持每类 precision、recall、F1-score；
7. 支持不同模型对比，例如 ResNet-18、MobileNetV2、SimpleCNN；
8. 支持 CPU 快速调试模式；
9. 支持 GitHub Actions 或简单自动测试；
10. README 中包含完整复现实验命令。

十五、验收标准

项目验收时，至少需要证明：

1. 代码仓库可以正常克隆；
2. 依赖可以正常安装；
3. 训练命令可以运行；
4. 测试命令可以运行；
5. 日志文件可以查看；
6. 模型权重可以保存和加载；
7. 输出中包含测试准确率；
8. 输出中包含混淆矩阵；
9. README 能指导他人复现实验；
10. vibe coding 过程记录完整。

十六、注意事项

1. 本任务重点不是追求最高准确率，而是完成一个规范、可运行、可复现的深度学习工程项目。
2. 训练轮数可以根据电脑性能调整。
3. 如果电脑性能较弱，可以先使用 1-2 个 epoch 跑通流程。
4. 如果没有 GPU，可以使用较小 batch size 和较少 epoch。
5. 不要把数据集、虚拟环境、大模型权重文件提交到 Git 仓库。
6. 不要把 W&B API Key 写进代码。
7. 所有重要修改都应通过 Git commit 记录。
8. 所有 AI 生成代码都需要自己运行验证，不能只提交未运行的代码。